

Documentação Técnica — Grade Horária IMP

Versão: 1.0

Data: 2026-05-20

Repositório: <https://github.com/HenriqueDantas2024/grade-horaria-imp>

Produção: <https://grade-horaria-imp.vercel.app>

Sumário

- 1. Visão Geral da Arquitetura
- 2. Stack Tecnológica
- 3. Estrutura de Pastas
- 4. Banco de Dados
- 5. Autenticação e Controle de Acesso
- 6. API Routes
- 7. Variáveis de Ambiente
- 8. GitFlow e Deploy
- 9. Componentes Reutilizáveis
- 10. Hooks Customizados
- 11. Decisões Técnicas Relevantes

1. Visão Geral da Arquitetura



O sistema é uma aplicação Next.js 16 com App Router. Todas as páginas do dashboard são Client Components ("use client") que se comunicam diretamente com o Supabase via SDK JavaScript. As API

Routes (/app/api/) são executadas server-side e usam o Supabase SSR com cookies para autenticação segura.

2. Stack Tecnológica

Camada	Tecnologia	Versão
Framework	Next.js	16.2.6
Linguagem	TypeScript	^5
UI	React	19.2.4
Estilização	Tailwind CSS	^4
Ícones	Phosphor Icons	^2.1.10
Ícones secundários	Lucide React	^1.16.0
Notificações	Sonner	^2.0.7
Backend (BaaS)	Supabase	^2.106.0
Auth SSR	@supabase/ssr	^0.10.3
E-mail	Resend	^6.12.3
Utilitários CSS	clsx + tailwind-merge	latest
Hospedagem	Vercel	—
Versionamento	GitHub	—

3. Estrutura de Pastas

```
src/
├── app/
│   ├── api/
│   │   ├── enviar-tirinhas/route.ts # POST – envia grade semanal por e-mail
│   │   ├── enviar-pagamento/route.ts # POST – envia resumo de pagamento por e-mail
│   │   └── dashboard/
│   │       ├── DashboardCards.tsx # componente principal do painel
│   │       ├── layout.tsx # layout com AppLayout
│   │       ├── page.tsx # /dashboard
│   │       ├── professores/page.tsx
│   │       ├── materias/page.tsx
│   │       ├── turmas/
│   │       │   ├── page.tsx # listagem de turmas
│   │       │   ├── [id]/
│   │       │   │   ├── page.tsx
│   │       │   │   ├── TurmaDetailClient.tsx
│   │       │   │   ├── TabGrade.tsx # montagem da grade semanal
│   │       │   │   └── TabDisciplinas.tsx
│   │       ├── tirinhas/page.tsx
│   │       ├── pagamentos/page.tsx # acesso restrito: role=diretor
│   │       └── relatorios/page.tsx
│   ├── login/page.tsx
│   ├── globals.css
│   ├── layout.tsx # root layout com <Toaster>
│   ├── page.tsx # redirect para /dashboard
│   └── components/
```

```

■   ■■■ layout/
■   ■   ■■■ AppLayout.tsx           # sidebar + main wrapper
■   ■■■ ui/
■       ■■■ Button.tsx
■       ■■■ Input.tsx
■       ■■■ Modal.tsx
■       ■■■ PageHeader.tsx
■       ■■■ Skeleton.tsx
■■■ lib/
■   ■■■ email/
■   ■   ■■■ tirinha-template.ts      # template HTML das tirinhas
■   ■■■ hooks/
■   ■   ■■■ useUserRole.ts           # lê perfil do usuário (coordenador|diretor)
■   ■   ■■■ useCountUp.ts           # animação de contador numérico
■   ■■■ supabase/
■       ■■■ client.ts               # createClient para Client Components
■       ■■■ server.ts               # createClient para Server Components / API Routes
■■■ proxy.ts                        # proteção de rotas (equivalente ao middleware)
public/
■■■ imp_concursos_logo.png
■■■ foto-aguas-claras.jpeg
■■■ foto-imp-exterior.jpeg
docs/
■■■ asbuilt.md
■■■ manual-usuario.md
■■■ documentacao-tecnica.md
■■■ memoria/
    ■■■ schema.sql
    ■■■ moscow-pagamentos.md

```

4. Banco de Dados

Tabelas

professores

Coluna	Tipo	Descrição
id	uuid PK	gerado automaticamente
nome	text	nome completo
email	text unique	e-mail de contato
ativo	boolean	default true
valor_hora_aula	numeric(8,2)	valor R\$/hora para cálculo de pagamento
created_at	timestampz	

materias

Coluna	Tipo	Descrição
id	uuid PK	
nome	text	nome da disciplina
created_at	timestampz	

professor_materias (pivot)

Coluna	Tipo
professor_id	uuid FK → professores
materia_id	uuid FK → materias

turmas

Coluna	Tipo	Descrição		
id	uuid PK			
codigo	text unique	ex: PMSAC-4098		
concurso	text	nome do concurso		
local	text	cidade/unidade		
turno	text	M \	T \	N
data_inicio	date			
status	text	ativa \	encerrada	
created_at	timestampz			

semanas

Coluna	Tipo	Descrição	
id	uuid PK		
data_inicio	date	sempre segunda-feira	
data_fim	date	sempre sábado	
status	text	rascunho \	publicada
created_at	timestampz		

aulas

Coluna	Tipo	Descrição		
id	uuid PK			
semana_id	uuid FK → semanas			
turma_id	uuid FK → turmas			
professor_id	uuid FK → professores			
materia_id	uuid FK → materias			
dia_semana	text	segunda...sabado		
turno	text	M \	T \	N
horario_inicio	time			
horario_fim	time			
carga_horaria	numeric(4,2)	horas da aula		
realizada	boolean	default false		
created_at	timestampz			

perfis

Coluna	Tipo	Descrição	
id	uuid PK FK → auth.users	mesmo ID do usuário Supabase Auth	
role	text	coordenador \	diretor
created_at	timestampz		

pagamentos

Coluna	Tipo	Descrição	
id	uuid PK		
professor_id	uuid FK → professores		
periodo_inicio	date	início do período	
periodo_fim	date	fim do período	
tipo	text	semanal \	mensal
total_horas	numeric(6,2)		
valor_hora	numeric(8,2)	snapshot do valor na época	
total_valor	numeric(10,2)	total_horas × valor_hora	
status	text	pendente \	pago
created_at	timestampz		

Row Level Security (RLS)

Todas as tabelas têm RLS habilitado. A política padrão para leitura e escrita exige `auth.role() = 'authenticated'`.

A tabela pagamentos tem política adicional:

```
CREATE POLICY "somente_diretor" ON pagamentos
  FOR ALL USING (
    EXISTS (
      SELECT 1 FROM perfis
      WHERE id = auth.uid() AND role = 'diretor'
    )
  );
```

Trigger automático de perfil

```
CREATE OR REPLACE FUNCTION handle_new_user()
  RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
  INSERT INTO perfis (id, role) VALUES (NEW.id, 'coordenador')
  ON CONFLICT DO NOTHING;
  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION handle_new_user();
```

Todo novo usuário criado recebe automaticamente o perfil coordenador. Para elevar a diretor, atualizar manualmente via SQL:

```
UPDATE perfis SET role = 'diretor' WHERE id = '<user_uuid>';
```

5. Autenticação e Controle de Acesso

Fluxo de autenticação

1. Usuário faz login em /login via `supabase.auth.signInWithPassword()`
2. Supabase retorna JWT armazenado em cookie `HttpOnly`
3. `proxy.ts` (equivalente ao `middleware.ts` no Next.js 16) verifica o cookie em todas as rotas `/dashboard/*`
4. Rotas não autenticadas são redirecionadas para /login

Controle de perfil (role-based)

- `useUserRole()` hook lê a tabela perfis e retorna `{ role, loading }`
- `AppLayout.tsx` filtra os itens do menu de acordo com o role
- `pagamentos/page.tsx` tem guarda de rota adicional:

```
`typescript
useEffect(() => {
  if (!loading && role !== "diretor") router.replace("/dashboard");
}, [role, loading]);
if (loading || role !== "diretor") return <spinner>;
`
```

- RLS no banco como terceira camada de proteção

Nota sobre Next.js 16

O Next.js 16 não suporta `middleware.ts` da forma convencional. O arquivo `proxy.ts` na raiz do `src/` substitui essa funcionalidade para proteção de rotas.

6. API Routes

POST /api/enviar-tirinhas

Envia a grade semanal por e-mail para um ou mais professores.

Body:

```
{
  "semana_inicio": "2026-05-18",
  "semana_fim": "2026-05-23",
  "professor_ids": ["uuid1", "uuid2"] // opcional – omitir envia para todos
}
```

Resposta:

```
{
  "enviados": 5,
  "falhas": 0,
  "detalhes": [
    { "professor": "João Silva", "email": "joao@...", "status": "ok" }
  ]
}
```

Funcionamento:

1. Autentica via Supabase SSR (cookies da requisição)
2. Busca semana pelo período informado
3. Para cada professor: busca aulas da semana + turmas/matérias relacionadas
4. Gera HTML personalizado via tirinha-template.ts
5. Envia via Resend

POST /api/enviar-pagamento

Envia resumo de pagamento por e-mail ao professor.

Body:

```
{
  "professor_id": "uuid",
  "periodo_inicio": "2026-05-18",
  "periodo_fim": "2026-05-23",
  "tipo": "semanal",
  "total_horas": 12.5,
  "valor_hora": 80.00,
  "total_valor": 1000.00
}
```

Resposta:

```
{ "ok": true }
```

7. Variáveis de Ambiente

Variável	Onde usar	Descrição
NEXT_PUBLIC_SUPABASE_URL	Client + Server	URL do projeto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Client + Server	Chave anônima pública do Supabase
RESEND_API_KEY	Somente Server (API Routes)	Chave da API Resend para envio de e-mails

> Arquivo .env.local para desenvolvimento local. Na Vercel, configurar em Settings → Environment Variables.

8. GitFlow e Deploy

Branches

Branch	Propósito
dev	desenvolvimento ativo
hml	homologação / testes
main	produção (deploy automático na Vercel)

Fluxo

feature → dev → (PR) → hml → (aprovação) → main → Vercel deploy automático

Deploy

A Vercel monitora a branch main. Todo push dispara build automático (~1-2 minutos). Build usa next build sem flags especiais.

9. Componentes Reutilizáveis

Componente	Localização	Props principais		
Button	ui/Button.tsx	variant (default\	ghost), size (sm\	md), disabled
Input	ui/Input.tsx	label, type, value, onChange, placeholder		
Modal	ui/Modal.tsx	open, onClose, title, children		
PageHeader	ui/PageHeader.tsx	title, description, action (ReactNode)		
Skeleton	ui/Skeleton.tsx	base shimmer div		
SkeletonTable	ui/Skeleton.tsx	rows, cols		
SkeletonKpi	ui/Skeleton.tsx	grid de 4 cards KPI		

Design Tokens (CSS variables em globals.css)

```
--color-navy          /* #1A1F36 – cor principal */
--color-primary       /* #E8193C – vermelho IMP */
--color-background    /* fundo da app */
--color-surface       /* fundo de cards/tabelas */
--color-border        /* bordas */
--color-text-primary
--color-text-secondary
--color-text-muted
--color-navy-medium   /* bordas da sidebar */
```

10. Hooks Customizados

useUserRole()

```
// src/lib/hooks/useUserRole.ts
// Retorna: { role: "coordenador" | "diretor" | null, loading: boolean }
```

Lê a tabela perfis para o auth.uid() atual. Usado em AppLayout e pagamentos/page.tsx.

useCountUp(target, duration, enabled)

```
// src/lib/hooks/useCountUp.ts
// Anima um número de 0 até `target` em `duration` ms com ease-out cúbico
// `enabled` permite aguardar carregamento dos dados antes de iniciar
```

Usado no Dashboard (KPIs) e Pagamentos (totais).

11. Decisões Técnicas Relevantes

Por que proxy.ts em vez de middleware.ts?

O Next.js 16.2.6 introduziu breaking changes no sistema de middleware. A abordagem com proxy.ts na raiz do src/ garante proteção de rotas sem depender da API de middleware, que mudou de comportamento nessa versão.

Por que Client Components em vez de Server Components?

As páginas do dashboard fazem múltiplas queries condicionais e têm interações ricas (modais, formulários, toggles). Server Components trazem benefício menor neste cenário, enquanto Client Components simplificam o estado e a reatividade.

Por que Promise.all() nas queries?

Múltiplas queries independentes no Supabase são executadas em paralelo, reduzindo o tempo de carregamento das páginas ao tempo da query mais lenta, em vez da soma de todas.

Por que Sonner em vez de alert()/confirm()?

alert() e confirm() bloqueiam o thread principal e têm aparência nativa inconsistente. Sonner oferece notificações não bloqueantes com suporte a richColors, closeButton e ações customizadas — alinhado ao padrão de UX do sistema.

Click-to-confirm para exclusão

Em vez de window.confirm(), o padrão adotado é: primeiro clique acende o botão vermelho + exibe toast de aviso; segundo clique (dentro de 3s) executa a exclusão. Evita exclusões acidentais sem bloquear a UI.

Next.js <Image> para o hero slideshow

As fotos do hero eram servidas como puro, fazendo o browser baixar o JPEG completo (~90KB cada). Com <Image fill sizes="100vw" priority>, o Next.js serve WebP otimizado no tamanho exato da viewport, com lazy loading automático nas fotos não visíveis.